



Security Report for Flowdometer - Time Tracking and SLAs for Stages

The below report outlines vulnerability and informational findings that were discovered via a mixture of automated scanning and manual security penetration testing. It is possible that there are other vulnerabilities that were not discovered during testing and are therefore not included on this report. Not all instances of each vulnerability type were enumerated.

The attack vector and evidence of vulnerability associated with each item is listed with a description of the vulnerability. Suggested remediation steps and additional educational resources for each class of vulnerability are also provided.

Please note that this is not a comprehensive list of vulnerabilities in your application. Similar and additional vulnerabilities outside of this report may exist. Please look through your entire codebase for additional security issues.

Contents

1. Unauthorized use of Session id Vulnerability

1. pages\SessionId.page, classes\Metadataservice.cls, classes\MetaDataUtilityCls.cls
2. pages\SessionId.page, classes\Metadataservice.cls, classes\MetaDataUtilityCls.cls

Unauthorized use of Session id

Finding 1 of 2

File

pages\SessionId.page

Code

```
<apex:page >
    Start_Of_Session_Id{!$Api.Session_ID}End_Of_Session_Id
</apex:page>
```

File

classes\MetadataService.cls

Code

```
...
private static String apiEnabledSessionId(){
    PageReference sessionPage=Page.SessionId;
    String vfContent=sessionPage.getContent().toString();
    Integer startIndex=vfContent.indexOf('Start_Of_Session_Id') + 'Start_Of_Session_Id'.length();
    Integer endIndex=vfContent.indexOf('End_Of_Session_Id');

    return vfContent.substring(startIndex, endIndex);
}
...

public static MetadataService.MetadataPort createService(){
    MetadataService.MetadataPort service=new MetadataService.MetadataPort();
    service.SessionHeader=new MetadataService.SessionHeader_element();
    //service.SessionHeader.sessionId=UserInfo.getSessionId();
    //service.SessionHeader.sessionId='!$Credential.OAuthToken';
    service.SessionHeader.sessionId=MetadataService.apiEnabledSessionId();
    return service;
}
...
```

File

classes\MetaDataUtilityCls.cls

Code

```
private static final String FLOWDOMETER_PERMISSION_SET_NAME = 'Flowdrometer__Flowdrometer_User';
...
public static void grantFieldEditAccess(
    String objectApiName,
    String fieldApiName
){
    //permission set, objectAPINameWLookup, LookupFieldAPIName
    try {
        // Check if retry is enabled via feature flag
        Boolean useRetry = FeatureFlagUtil.isEnabled(FeatureFlagUtil.FEATURE_RETRY_METADATA_OPERATIONS);

        if (useRetry) {
            // Use the retry mechanism for this operation
            retryOperation(new GrantFieldAccessOperation(objectApiName, fieldApiName), 3);
        } else {
            // Use the direct method without retry
            MetadataService.MetadataPort service = MetadataService.createService();
            service.timeout_x = 180000; // Increase timeout to 3 minutes
            MetadataService.PermissionSet permissionSet = (MetadataService.PermissionSet)
```

Notes

In the reported instance application is using current users sessionId to UPDATE metadata like permissionSet which is against Salesforce Security guidelines. Creating or Updating metadata is not allowed. Please obtain an OAuth token to make the API call rather than getting the logged-in user's session Id.

Finding 2 of 2

File

pages\SessionId.page

Code

```
<apex:page >
    Start_Of_Session_Id{!$Api.Session_ID}End_Of_Session_Id
</apex:page>
```

File

classes\MetadataService.cls

Code

```
private static String apiEnabledSessionId(){
    PageReference sessionPage=Page.SessionId;
    String vfContent=sessionPage.getContent().toString();
    Integer startIndex=vfContent.indexOf('Start_Of_Session_Id') + 'Start_Of_Session_Id'.length();
    Integer endIndex=vfContent.indexOf('End_Of_Session_Id');

    return vfContent.substring(startIndex, endIndex);
}

...

public static MetadataService.MetadataPort createService(){
    MetadataService.MetadataPort service=new MetadataService.MetadataPort();
    service.SessionHeader=new MetadataService.SessionHeader_element();
    //service.SessionHeader.sessionId=UserInfo.getSessionId();
    //service.SessionHeader.sessionId='{!$Credential.OAuthToken}';
    service.SessionHeader.sessionId=MetadataService.apiEnabledSessionId();
    return service;
}

...
```

File

classes\MetaDataUtilityCls.cls

Code

```
private static final String FLOWDOMETER_PERMISSION_SET_NAME = 'Flowdometer__Flowdometer_User';
...

public Object execute() {
    MetadataService.MetadataPort service = MetadataService.createService();
    service.timeout_x = 180000; // Increase timeout to 3 minutes (from 2 minutes)
    MetadataService.PermissionSet permissionSet = (MetadataService.PermissionSet) service.readMetadata(
        'PermissionSet',
        new List<String>{ FLOWDOMETER_PERMISSION_SET_NAME }
    )
    .getRecords()[0];
    MetadataService.PermissionSetFieldPermissions fieldPermission = new
MetadataService.PermissionSetFieldPermissions();
    fieldPermission.field = objectApiName + '.' + fieldApiName;
    fieldPermission.editable = true;
    fieldPermission.readable = true;
    permissionSet.fieldPermissions.add(fieldPermission);

    List<MetadataService.SaveResult> results = service.updateMetadata(
        new List<MetadataService.Metadata>{ permissionSet }
    );
    ...
}
```

Notes

In the reported instance application is using current users sessionId to UPDATE metadata like permissionSet which is against Salesforce Security guidelines. Creating or Updating metadata is not allowed. Please obtain an OAuth token to make the API call rather than getting the logged-in user's session Id.

Report provided by Salesforce Product Security and does not represent in any way a guarantee of security

Information contained in this report may be confidential

Generated 12/05/2025 at 08:10 PM

